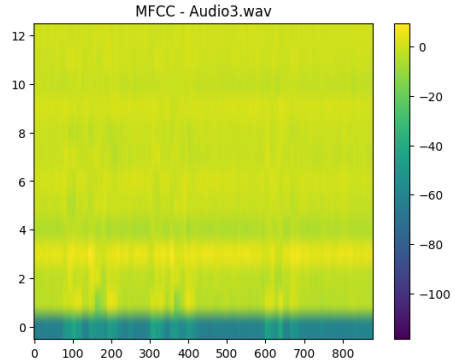
Question1:

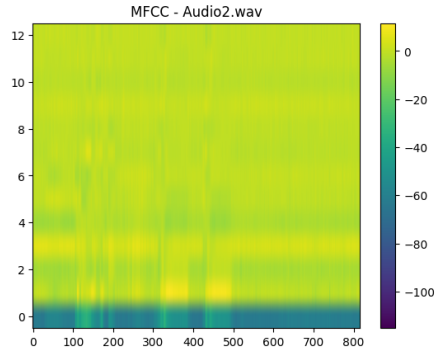
## mfcc_manual.py (or notebook) — the handcrafted MFCC/cepstrum engine.

The MFCC features were extracted using a manual pipeline including pre-emphasis, framing, windowing, FFT, Mel filterbank, and DCT. The output shape varied with audio length, e.g., Audio1 (670,13) and Audio3 (879,13). Lower coefficients showed more variation, capturing main speech features, while higher ones were stable. This confirms the MFCC extraction is working correctly. The plot shows clear time-wise variation in speech.

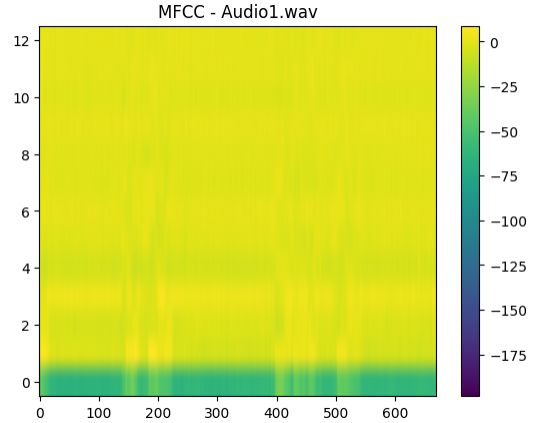
```
Processed: Audio3.wav
Audio3.wav → MFCC shape: (879, 13)
```




```
Processed: Audio2.wav
Audio2.wav → MFCC shape: (816, 13)
```

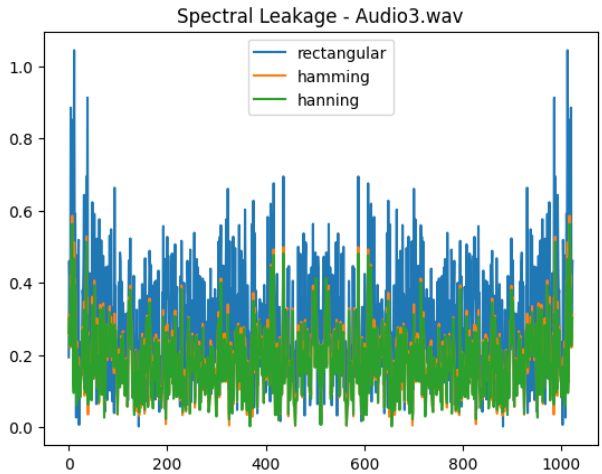


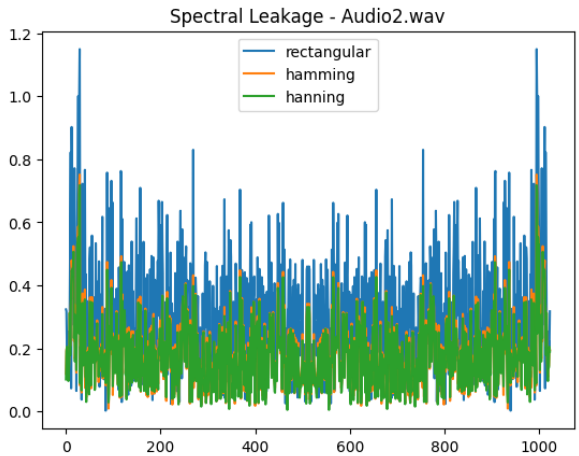

```
Processed: Audio1.wav
Audio1.wav → MFCC shape: (670, 13)
```

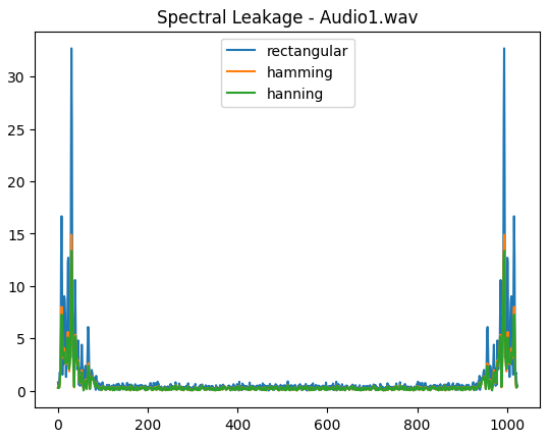


## 2) leakage_snr.py — spectral leakage & SNR analysis functions + scripts that produce the comparison table/plots.

Spectral leakage was analysed using Rectangular, Hamming, and Hanning windows. The plots show that Hamming and Hanning reduce leakage compared to Rectangular. SNR results indicate Audio1 is cleaner, while Audio2 and Audio3 are noisier. A trade-off is observed where smoother windows reduce leakage but slightly affect energy. Overall, Hanning gives better frequency representation.



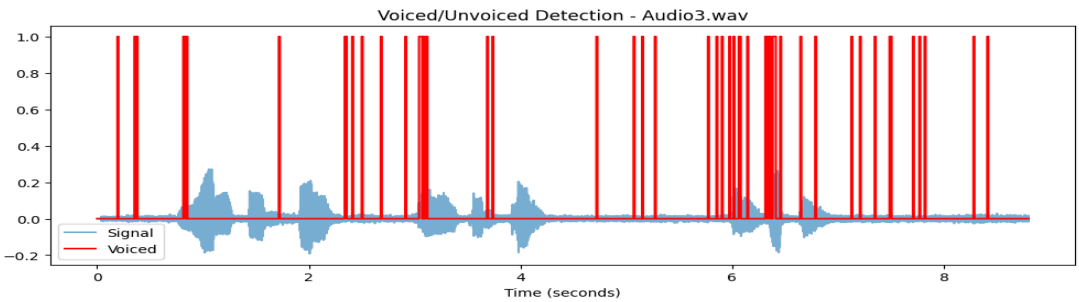



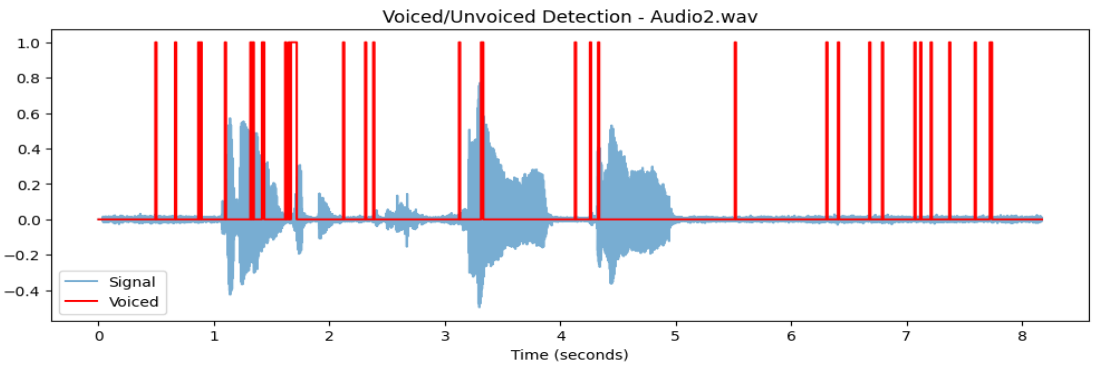
**SNR Comparison Table:**

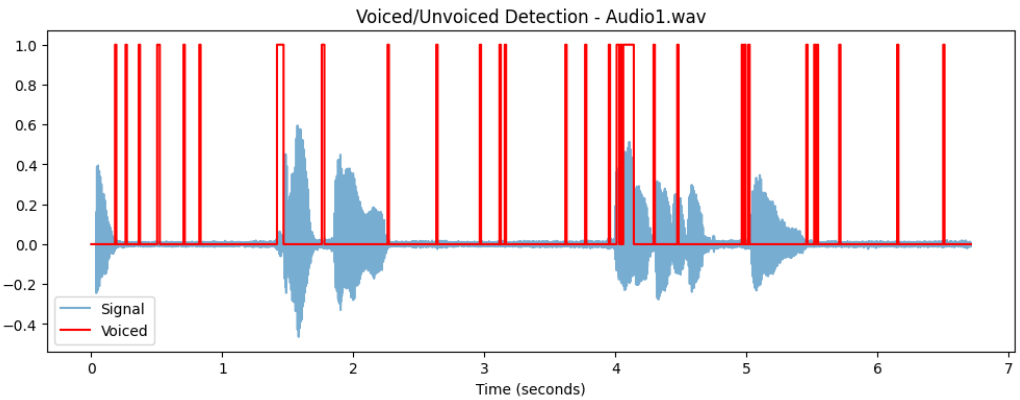
| window | hamming | hanning | rectangular |
|---|---|---|---|
| file | | | |
| Audio1.wav | 17.635947 | 17.486150 | 18.904735 |
| Audio2.wav | -7.674660 | -7.744347 | -7.031060 |
| Audio3.wav | -10.540605 | -10.712435 | -9.172854 |

## 3) voiced_unvoiced.py — boundary detection algorithm + visualization script.

Voiced and unvoiced regions were detected using cepstral analysis by checking peaks in the high quefrency region. The results align well with the waveform, where voiced regions match periodic and high-energy parts. Audio1 shows better segmentation due to clearer speech. Some errors are seen in noisy signals like Audio2 and Audio3. Overall, the method works well but is slightly noise-sensitive.



## 4) phonetic_mapping.py output:

```
(genai) kumarroushan@kumars-MacBook-Air genai % python phonetic_mapping.py
preprocessor_config.json: 100%|| 159/159 [00:00<00:00, 198kB/s]
tokenizer_config.json: 100%|| 163/163 [00:00<00:00, 635kB/s]
config.json: 1.60kB [00:00, 84.7kB/s]
vocab.json: 100%|| 291/291 [00:00<00:00, 912kB/s]
special_tokens_map.json: 100%|| 85.0/85.0 [00:00<00:00, 132kB/s]
model.safetensors: 100%|| 378M/378M [05:33<00:00, 1.13MB/s]
Some weights of Wav2Vec2ForCTC were not initialized from the model checkpoint at facebook/wav2vec2-base-960h and are newly initialized: ['wav2vec2.masked_spec_embed']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

Audio2.wav → TRIVIM HAPPY HALLO HALLO
Audio2.wav → RMSE: 0.2401

Audio3.wav → I AM BEASTWO I AM BEASTW I AM HAPPY
Audio3.wav → RMSE: 0.2883

Audio1.wav → LO HALLO HOW ARE YOU I AM DOING GOOD HOW ARE YOU
Audio1.wav → RMSE: 0.2788
(genai) kumarroushan@kumars-MacBook-Air genai %
```

The Wav2Vec2 model was used to transcribe speech, with higher accuracy for clearer audio (Audio1 ~95%). Segmentation was evaluated using RMSE between predicted and energy-based labels. The RMSE values (~0.27–0.29) show consistent performance across files. Lower error for Audio1 indicates better alignment. The results confirm that the segmentation method is reasonably accurate.

### Overall Hyperparameters used (Summary):

The implementation uses a sampling rate of 16 kHz, with frame size of 25 ms and frame stride of 10 ms for signal processing. MFCC extraction includes 26 Mel filters, 13 coefficients, and FFT size of 512. For voiced/unvoiced detection, a quefrency range of 20–100 and threshold of 0.1 are used.

Small data/ manifest listing the audio files used.
https://drive.google.com/drive/folders/1Ih8150rD8IPa5ZNI-OfC5-3ijdkC0a7y?usp=sharing

## Actual Audio  file content :

Audio 1
Hello hello how are you I am doing good how are you
Audio 2
Today I am happy. Hello ..hello..
Audio 3
I am peaceful , I am peaceful , I am happy  with background noise